COMPUTING 2 - PARALLEL COMPUTING

Chapter 6 Mock Test

OpenMP, MPI, and Hybrid Programming

Suggested time	Total points	Coverage
55 minutes	60	Fork-join, worksharing, data sharing, reduction, scheduling, tasks, MPI collectives, hybrid design

NAME		STUDENT ID	
DATE		SCORE	/ 60

Instructions
- Answer every question and show essential reasoning for partial credit.
- For true/false items, an unsupported letter receives no credit.
- Unless stated otherwise, assume NVIDIA warp size 32 and ideal arithmetic.
- Keep the answer section closed until the suggested time has elapsed.

Questions

6.1 Repair an OpenMP reduction — 8 points

Identify the race and write an idiomatic repair.

```
#pragma omp parallel for
for (int i=0; i<n; ++i)
    sum += a[i];
```

WORK AREA

6.2 Data-sharing attributes — 7 points

For the loop below under default(none), classify A, x, y, m, n, i, j, and sum as shared or private and write the directive.

```
for (int i=0; i<m; ++i) {
    double sum = 0.0;
    for (int j=0; j<n; ++j) sum += A[i*n+j] * x[j];
    y[i] = sum;
}
```

WORK AREA

6.3 Static versus dynamic scheduling — 7 points

Iteration i has cost proportional to i. Compare schedule(static) and schedule(dynamic,1). Which likely balances better, and what overhead does it add? Name a better compromise than chunk size 1.

WORK AREA

6.4 Synchronization choice — 7 points

Choose reduction, atomic, critical, barrier, or no synchronization for: (a) scalar sum, (b) one supported counter increment, (c) update of a multi-statement non-atomic data structure, (d) phase boundary, (e) independent writes y[i].

WORK AREA

6.5 OpenMP false sharing — 7 points

Threads accumulate into partial[tid] inside a long loop. Explain when false sharing appears and give a lower-overhead repair.

WORK AREA

6.6 Task dependencies — 8 points

Three OpenMP tasks compute A, then B from A, and C from A; a final task D uses B and C. Express the dependency graph and state which tasks may run concurrently.

WORK AREA

6.7 MPI collective selection — 8 points

Choose the best collective: (a) root sends a full x to all ranks, (b) root sends distinct equal row blocks, (c) ranks combine scalar partial sums at root, (d) every rank needs every rank's equal-size vector segment, (e) root collects output segments.

WORK AREA

6.8 Deadlock and hybrid design — 8 points

All ranks execute blocking MPI_Send to rank+1 before MPI_Recv from rank-1 in a ring. Explain the deadlock risk, give two repairs, and state one reason to use MPI+OpenMP on a cluster node.

WORK AREA

ANSWER SECTION

Chapter 6: Answer Key & Worked Solutions

Grade the method as well as the final result. Numerical answers show enough intermediate work to identify where an error occurred.

Quick Answer Key

Q	Topic	Short answer
6.1	Repair an OpenMP reduction	sum is raced; use reduction(+:sum)
6.2	Data-sharing attributes	shared(A,x,y,m,n); i private by worksharing; j and sum block-local private
6.3	Static versus dynamic scheduling	Dynamic balances irregular cost but adds runtime scheduling; use guided or a larger dynamic chunk
6.4	Synchronization choice	reduction; atomic; critical/lock; barrier; none
6.5	OpenMP false sharing	Adjacent partials share a line; private accumulator plus one final write/reduction
6.6	Task dependencies	A -> {B,C} -> D; B and C may run concurrently
6.7	MPI collective selection	Bcast, Scatter, Reduce, Allgather, Gather
6.8	Deadlock and hybrid design	Circular wait; use Sendrecv or nonblocking Irecv/Isend+Waitall; hybrid reduces ranks and exploits shared memory within a node

Worked Solutions

6.1 Repair an OpenMP reduction — 8 points

Answer. Every thread performs a read-modify-write on the same variable sum.

```
#pragma omp parallel for reduction(+:sum)
for (int i=0; i<n; ++i)
    sum += a[i];
```

Common trap. critical or atomic can be correct, but reduction usually avoids serializing every iteration.

6.2 Data-sharing attributes — 7 points

Answer. Input/output arrays and loop bounds are shared. Variables declared inside a loop iteration are private to that iteration.

```
#pragma omp parallel for default(none) shared(A,x,y,m,n)
for (int i=0; i<m; ++i) {
    double sum = 0.0;
    for (int j=0; j<n; ++j) sum += A[i*n+j] * x[j];
    y[i] = sum;
}
```

6.3 Static versus dynamic scheduling — 7 points

Answer. Contiguous static chunks give later threads systematically more work in this cost pattern.

- dynamic,1 redistributes work as threads finish and therefore balances well.
- It performs frequent runtime scheduling and can harm locality.
- schedule(guided) or dynamic with a larger chunk often balances overhead and irregularity.

6.4 Synchronization choice — 7 points

Answer. Use the least restrictive primitive that expresses the actual dependency.

- (a) reduction, (b) atomic, (c) critical or lock, (d) barrier, (e) no synchronization when each i is uniquely owned.

6.5 OpenMP false sharing — 7 points

Answer. Frequently written per-thread slots can share one cache line even though their indices differ.

- Accumulate in a local scalar, which normally remains in a register.
- Write partial[tid] once after the loop, or use an OpenMP reduction.
- Padding is another option when the array itself must be updated frequently.

6.6 Task dependencies — 8 points

Answer. The graph has one fork after A and one join before D.

```
#pragma omp task depend(out:A)        // produce A
#pragma omp task depend(in:A) depend(out:B)  // B
#pragma omp task depend(in:A) depend(out:C)  // C
#pragma omp task depend(in:B,C) depend(out:D) // D
```

Common trap. Task creation order alone is not a portable substitute for depend clauses when execution dependencies matter.

6.7 MPI collective selection — 8 points

Answer. The result placement and whether contributions are equal or distinct determine the collective.

- (a) MPI_Bcast, (b) MPI_Scatter, (c) MPI_Reduce, (d) MPI_Allgather, (e) MPI_Gather.
- Use the *v variants when counts differ by rank.

6.8 Deadlock and hybrid design — 8 points

Answer. If sends wait for matching receives, every rank can wait while no receive has been posted.

- Repair with MPI_Sendrecv.
- Or post MPI_Irecv before MPI_Isend and finish with MPI_Waitall.
- Hybrid programming uses MPI across nodes and threads within a shared-memory node, often reducing duplicated data and message count.

Common trap. Some small eager sends may appear to work, but correctness must not depend on buffering behavior.